

PROJECT: Install Jenkins inside the Docker container and install plugins inside of it and try to build the application and Hosted in a container.

Definition: This project involves setting up a Continuous Integration and Continuous Deployment (CI/CD) pipeline by installing Jenkins inside a Docker container, configuring essential plugins, and automating the process of building and deploying an application. Jenkins is used to pull the source code from a repository, build it using appropriate tools, and create a Docker image of the application. The application is then deployed and hosted as a running container, ensuring consistency across environments. This approach leverages containerization for portability and scalability while enabling automated, efficient, and reliable software delivery.

Scope of this project: This project involves setting up a Continuous Integration and Continuous Deployment (CI/CD) pipeline by installing Jenkins inside a Docker container, configuring essential plugins, and automating the process of building and deploying an application. Jenkins is used to pull the source code from a repository, build it using appropriate tools, and create a Docker image of the application. The application is then deployed and hosted as a running container, ensuring consistency across environments. This approach leverages containerization for portability and scalability while enabling automated, efficient, and reliable software delivery.

Methodologies:

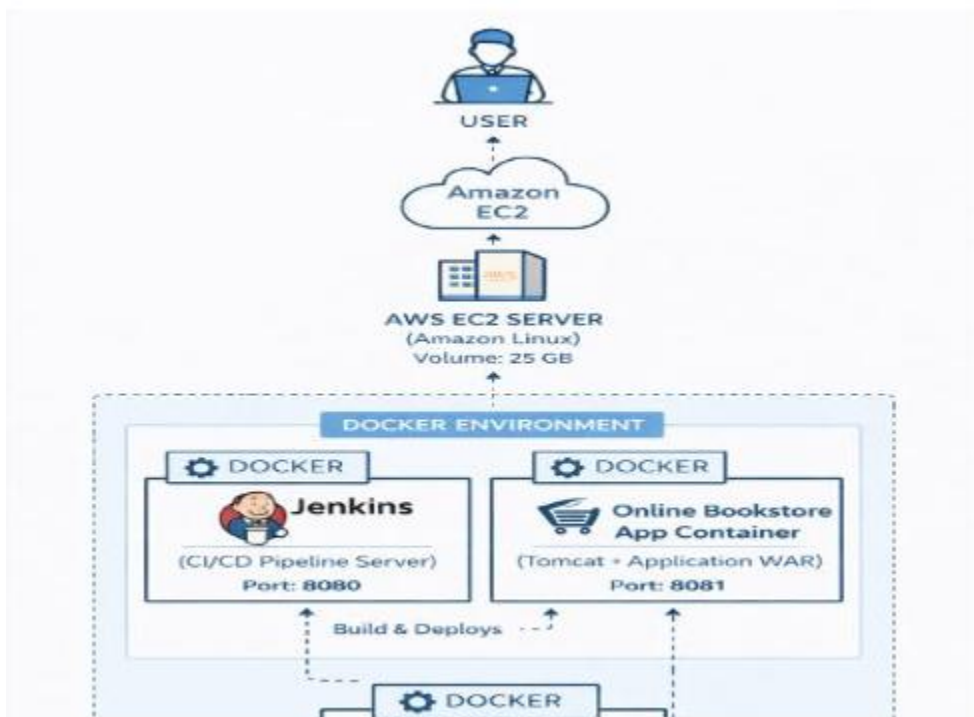
1. Set up Docker environment on the system.
2. Run Jenkins inside a Docker container.
3. Access Jenkins and perform initial configuration.
4. Install required Jenkins plugins (Git, Docker, Maven, Pipeline).
5. Connect Jenkins to source code repository (e.g., GitHub).
6. Create and configure a CI/CD pipeline job in Jenkins.
7. Pull the application source code from the repository.
8. Build the application using build tools (Maven/Gradle).
9. Create a Docker image for the application.
10. Run the Docker container to deploy (host) the application.
11. Test the deployed application in the browser.
12. Automate the process to trigger builds on code changes.

Services used in this Project:

Here are the services/tools used in this project in simple lines:

1. Docker – to create and run containers for Jenkins and the application.
2. Jenkins – to automate build and deployment (CI/CD pipeline).
3. Git – to manage source code.
4. GitHub – to host the application repository.
5. Maven / Gradle – to build the application.
6. Docker Hub – to store and pull Docker images.
7. Docker Engine – to build and run Docker containers.

Architecture Diagram:



Implementation and Execution:

STEP 1: Launch a Server:

EC2 > Instances

Instances (1/1) Info

Connect Instance state Actions Launch instances

Saved filter sets Choose filter set Find Instance by attribute or tag (case-sensitive)

Name	Instance ID	Instance state	Instance type	Status check	Alarm status	Availabil
docker-jenkins-server	i-081ce527f4908e947	Running	t3.micro	3/3 checks passed	View alarms +	ap-south

STEP 2: Update and Install Docker.

```
aws [Alt+S] Ask Amazon Q Asia Pacific (Mumbai) Shreenivasa (0741-3904-  
~#  
~### Amazon Linux 2023  
~\#####\  
~~\####\  
~~\##|  
~~\$/ https://aws.amazon.com/linux/amazon-linux-2023  
~~V~' ->  
~~~~  
~~~~.  
~~~~.-  
~~~~m/  
  
Last login: Mon Apr 13 12:09:56 2026 from 13.233.177.5  
[ec2-user@ip-172-31-37-175 ~]$ sudo su  
[root@ip-172-31-37-175 ec2-user]# yum update -y  
Last metadata expiration check: 3:06:44 ago on Mon Apr 13 12:06:29 2026.  
Dependencies resolved.  
Nothing to do.  
Complete!  
[root@ip-172-31-37-175 ec2-user]# docker --version  
Docker version 25.0.14, build 0bab007  
[root@ip-172-31-37-175 ec2-user]#
```

STEP 4: Start the Docker

```

aws [Alt+S] Ask Amazon Q Asia Pacific (Mumbai) Shreenivasa [0741-3904-4318] Shreenivasa
#
####
~\#####
~\#####
~\#####
~\#/? https://aws.amazon.com/linux/amazon-linux-2023
~\V~' ->
~\m/
Last login: Mon Apr 13 12:09:56 2026 from 13.233.177.5
[ec2-user@ip-172-31-37-175 ~]$ sudo su
[root@ip-172-31-37-175 ec2-user]# yum update -y
Last metadata expiration check: 3:06:44 ago on Mon Apr 13 12:06:29 2026.
Dependencies resolved.
Nothing to do.
Complete!
[root@ip-172-31-37-175 ec2-user]# docker --version
Docker version 25.0.14, build 0bab007
[root@ip-172-31-37-175 ec2-user]# ^C
[root@ip-172-31-37-175 ec2-user]# systemctl start docker
[root@ip-172-31-37-175 ec2-user]#

```

STEP 5: Pulled the Docker image.

```
aws [Alt+S] Ask Amazon Q Asia Pacific (Mumbai) Shreenivasa (0741-3904-431) Shreen
```

```
Commands:
build      Build an image from a Dockerfile
history    Show the history of an image
import     Import the contents from a tarball to create a filesystem image
inspect    Display detailed information on one or more images
load       Load an image from a tar archive or STDIN
ls         List images
prune      Remove unused images
pull       Download an image from a registry
push       Upload an image to a registry
rm         Remove one or more images
save       Save one or more images to a tar archive (streamed to STDOUT by default)
tag        Create a tag TARGET_IMAGE that refers to SOURCE_IMAGE

Run 'docker image COMMAND --help' for more information on a command.
[root@ip-172-31-37-175 ec2-user]# docker images
REPOSITORY          TAG                 IMAGE ID            CREATED             SIZE
bookstore-app       latest             c87aa334f3db       About an hour ago  421MB
<none>              <none>             68530b96ee04       About an hour ago  421MB
<none>              <none>             5bc43a611530       2 hours ago       421MB
jenkins/jenkins     latest             79600f54e12d       6 days ago        482MB
hello-world         latest             e2ac70e7319a       2 weeks ago       10.1kB
jenkins/jenkins     lts               3e88dcb3d033       3 weeks ago       487MB
[root@ip-172-31-37-175 ec2-user]#
```

STEP 6: Create Jenkins Docker Volume and Run Jenkins Container.

```
aws | [AWS Logo] | Search | [Alt+S] | [Icons: Home, Notification, Help, Settings] | Asia Pacific (Mumbai) | Shreenivasa (0741-3904-4318) | Shreenivasa | [Close]
```

```
#  
#####  
~\#####\  
~~\####|  
~~\#/ https://aws.amazon.com/linux/amazon-linux-2023  
~~V~' '~>  
~~~~  
~~~~_  
~/m/'
```

```
Last login: Mon Apr 13 15:18:09 2026 from 13.233.177.3  
[ec2-user@ip-172-31-37-175 ~]$ sudo su  
[root@ip-172-31-37-175 ec2-user]# docker volume create jenkins-data  
jenkins-data  
[root@ip-172-31-37-175 ec2-user]#
```

STEP 7: Show the current running Docker Container.

[illegible]

STEP 8: Access the Jenkins Docker container's terminal (shell) to run commands inside the container.

```
aws [Alt+S] Search Asia Pacific (Mumbai) Shreenivasa (0741-3904-4318) Shreenivasa
```

```
~\#### Amazon Linux 2023
~~\#####\
~~\###|
~~\#/ https://aws.amazon.com/linux/amazon-linux-2023
~~v-' '->
~~~
~~~-
~/m/'

Last login: Mon Apr 13 15:18:09 2026 from 13.233.177.3
[ec2-user@ip-172-31-37-175 ~]$ sudo su
[root@ip-172-31-37-175 ec2-user]# docker volume create jenkins-data
jenkins-data
[root@ip-172-31-37-175 ec2-user]# ^C
[root@ip-172-31-37-175 ec2-user]# docker ps
CONTAINER ID        IMAGE               NAMES                  CREATED          STATUS             PORTS
1558cc4c15e5       jenkins/jenkins:lts "/usr/bin/tini -- /u... 2 hours ago      Up 2 hours         0.0.0.0:8080->8080/tcp, :::8080->8080/tcp, 0.0.0.0:50000->50000/tcp, :::50000->50000/tcp   jenkins
[root@ip-172-31-37-175 ec2-user]# ^C
[root@ip-172-31-37-175 ec2-user]# docker exec -it jenkins bash
root@1558cc4c15e5:/# java -jar /usr/share/jenkins/jenkins.war --version
2.541.3
root@1558cc4c15e5:/#
```

STEP 9: Access Jenkins Web Interface

← → ↻ ⚠ Not secure 13.233.186.33:8080/login?from=%2F ☆ 🔒 🔍 📄 🌐 ⋮



Sign in to Jenkins

Username

Password

☐ Keep me signed in

Sign in

 Jenkins

+ New Item

📁 Build History

🔍 ⚙️ 👤

Add description

Build Queue

No builds in the queue.

Build Executor Status

0/2

All +

S	W	Name ↓	Last Success	Last Failure	Last Duration
...	☀️	bookstore-project	N/A	N/A	N/A
✓	☁️	bookstore-project1	1 hr 25 min #12	1 hr 42 min #9	15 sec

Icon: S M L ...

STEP 10: Installed Additional Jenkins Plugins.

 Jenkins

Manage Jenkins ▾ / Plugins

🔍 ⚙️ 👤

Plugins

📄 Updates

📁 Available plugins

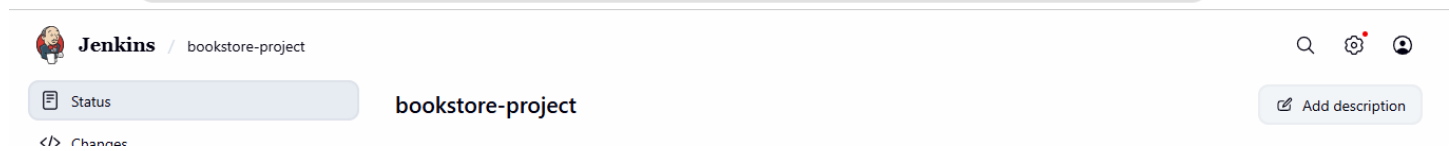
🔗 Installed plugins

⚙️ Advanced settings

🔍 maven

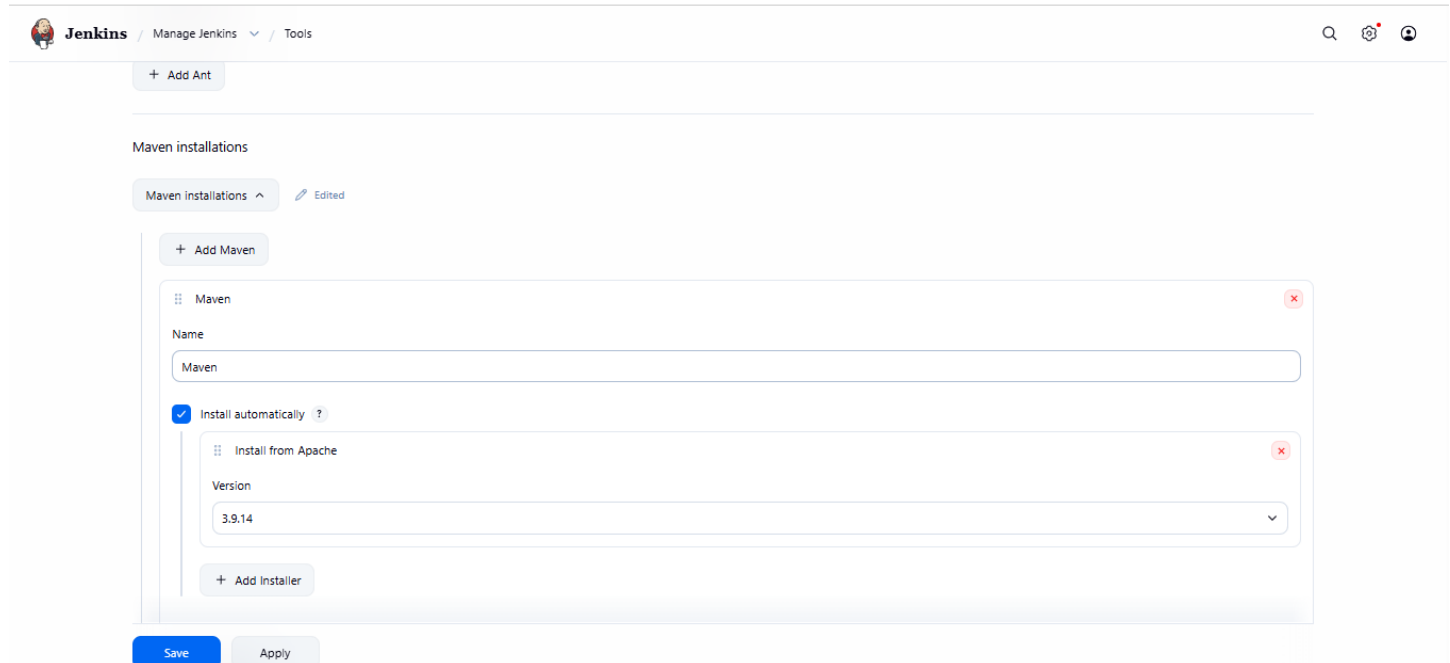
Name ↓	Health	Enabled
Maven Integration plugin 3.27 This plugin provides a deep integration between Jenkins and Maven. It adds support for automatic triggers between projects depending on SNAPSHOTS as well as the automated configuration of various Jenkins publishers such as Junit. Report an issue with this plugin	100	🔗

STEP 11: Create a Job.



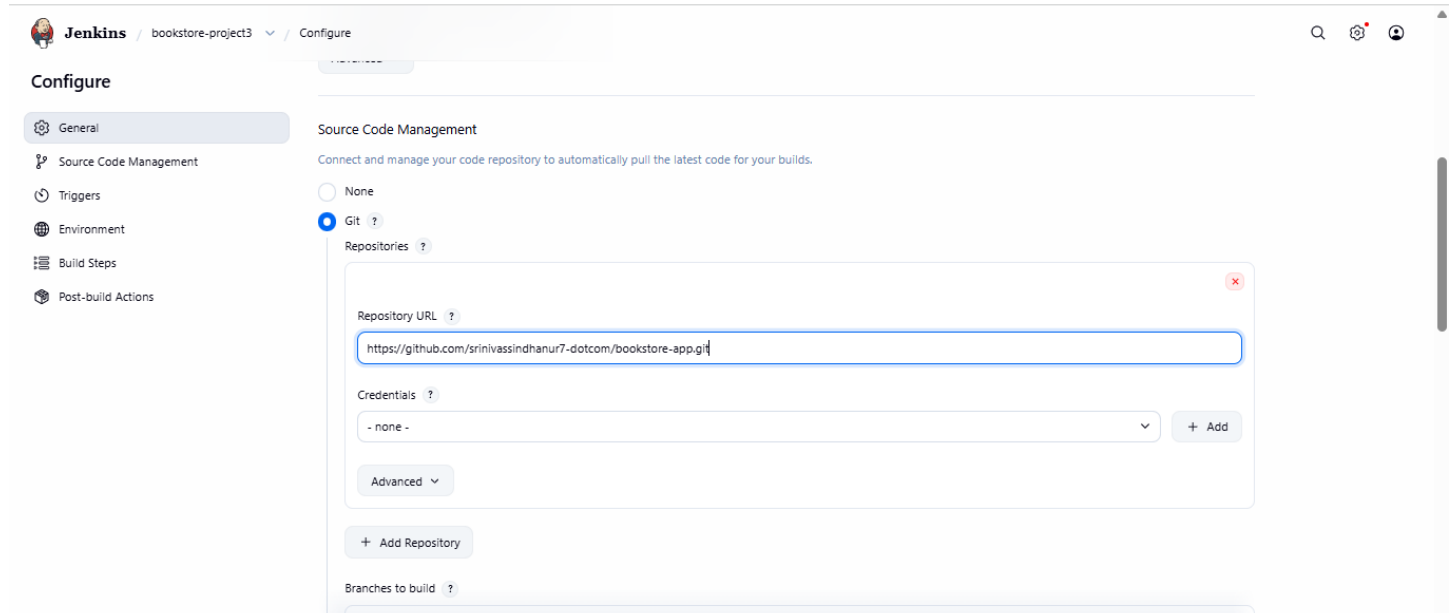
The screenshot shows the Jenkins web interface for a job named 'bookstore-project'. The breadcrumb navigation at the top reads 'Jenkins / bookstore-project'. Below the breadcrumb, there is a 'Status' button on the left and an 'Add description' button on the right. A 'Change' link is visible below the status button.

STEP 12: Add the maven path.



The screenshot shows the 'Tools' configuration page in Jenkins. The breadcrumb navigation is 'Jenkins / Manage Jenkins / Tools'. There is an 'Add Ant' button at the top. Under the 'Maven installations' section, there is an 'Add Maven' button. A modal form for adding a new Maven installation is open. It contains a 'Name' field with the value 'Maven', a checked 'Install automatically' checkbox, and an 'Install from Apache' section with a 'Version' dropdown menu set to '3.9.14'. At the bottom of the modal are 'Add Installer', 'Save', and 'Apply' buttons.

STEP 13: Clone Online Bookstore Application Repository.



The screenshot shows the 'Configure' page for a Jenkins job named 'bookstore-project3'. The breadcrumb navigation is 'Jenkins / bookstore-project3 / Configure'. On the left is a sidebar with configuration tabs: 'General', 'Source Code Management', 'Triggers', 'Environment', 'Build Steps', and 'Post-build Actions'. The 'Source Code Management' tab is selected. The main area shows options for 'Source Code Management' with 'None' and 'Git' (selected) radio buttons. Below, there is a 'Repositories' section with a 'Repository URL' field containing 'https://github.com/srinivassindhanur7-dotcom/bookstore-app.git', a 'Credentials' dropdown set to '- none -', and an 'Add' button. There is also an 'Advanced' dropdown and an 'Add Repository' button at the bottom.

STEP 14: Build Docker Image for Online Bookstore.

**Jenkins**

bookstore-project1

Status

</> Changes

Workspace

Build Now

Configure

Delete Project

Rename

bookstore-project1

Permalinks

- Last build (#12), 1 hr 33 min ago
- Last stable build (#12), 1 hr 33 min ago
- Last successful build (#12), 1 hr 33 min ago
- Last failed build (#9), 1 hr 51 min ago
- Last unsuccessful build (#9), 1 hr 51 min ago
- Last completed build (#12), 1 hr 33 min ago

Builds >

Filter

Today

 #12 1:59 pm

Search

Settings

Profile

Add description

 **Jenkins** / bookstore-project1  / #12 

```
temurin:17@sha256:2ff0b91aaa4cb5dc35b4760c37fe064390a486dcd90b361caa72b55d598601f3
#4 DONE 0.0s

#5 [internal] load build context
#5 transferring context: 211B done
#5 DONE 0.0s

#6 [2/3] WORKDIR /app
#6 CACHED

#7 [3/3] COPY target/bookstore-app-1.0-SNAPSHOT.jar app.jar
#7 CACHED

#8 exporting to image
#8 exporting layers done
#8 writing image sha256:c87aa354f3db955d5cb3889f5f6d3b1d71bae04202c1d6e6b648c6de7fb8416d done
#8 naming to docker.io/library/bookstore-app done
#8 DONE 0.0s

Finished: SUCCESS
```

STEP 15: Access the Application.

```
  Search [Alt+S]   ? ⚙️ Asia Pacific (Mumbai) ▾ Shreenivasa (0741-3904-4318) ▾ Shreenivasa
```

```
#  
##### Amazon Linux 2023  
~~~~~\#####  
~~~~~\#####|  
~~~~~\#/  
~~~~~V~' -> https://aws.amazon.com/linux/amazon-linux-2023  
  
~~~~~  
~~~~~-.-.  
~~~~~/m/'
```

Last login: Mon Apr 13 15:18:24 2026 from 13.233.177.3
[ec2-user@ip-172-31-37-175 ~]\$ sudo su
[root@ip-172-31-37-175 ec2-user]# docker logs bookstore-container
Hello Bookstore App Running Successfully!
[root@ip-172-31-37-175 ec2-user]# █

Conclusion: This project successfully demonstrates how to implement a complete CI/CD pipeline by integrating Jenkins with Docker. By running Jenkins inside a Docker container, the setup becomes portable, scalable, and easy to manage. The project automates key stages of the software development lifecycle, including code integration, build, and deployment, reducing manual effort and minimizing errors. Containerizing the application ensures consistency across different environments and simplifies deployment. Overall, this approach enhances efficiency, reliability, and speed in software delivery, making it highly suitable for modern DevOps practices.